BookScope API Documentation

BookScope API Documentation

Overview

BookScope API is a REST API for book catalog browsing, personal bookshelf management, reviews, explainable recommendations, and reading analytics.

Repository:

- https://github.com/IJF9GAWHY8FGA8/bookscope-api

Related submission assets:

- API documentation PDF: https://github.com/IJF9GAWHY8FGA8/bookscope-api/blob/main/docs/api_documentation.pdf
- Technical report PDF: https://github.com/IJF9GAWHY8FGA8/bookscope-api/blob/main/docs/technical_report.pdf
- Slides PPTX: https://github.com/IJF9GAWHY8FGA8/bookscope-api/blob/main/slides/bookscope_presentation.pptx

Base URLs

- Runtime API root: `/api/`
- Health endpoint: `/api/health/`
- Schema endpoint: `/api/schema/`
- Swagger UI: `/api/docs/`

Authentication

Authentication uses Bearer tokens via JSON Web Tokens.

Register

- `POST /api/auth/register/`

Example request:

```
{
  "username": "reader1",
  "email": "reader1@example.com",
  "password": "StrongPass123!"
}
```

Example response:

```
{
  "user": {
    "id": 1,
    "username": "reader1",
    "email": "reader1@example.com"
  },
  "tokens": {
    "access": "<jwt-access-token>",
    "refresh": "<jwt-refresh-token>"
  }
}
```

Obtain Access Token

- `POST /api/auth/token/`

Example request:

```
{
  "username": "reader1",
  "password": "StrongPass123!"
}
```

Refresh Token

- `POST /api/auth/token/refresh/`

Current User

- `GET /api/auth/me/`

Requires:

- `Authorization: Bearer `

Example response:

```
{
  "id": 1,
  "username": "reader1",
  "email": "reader1@example.com"
}
```

Catalog Endpoints

Books

- `GET /api/books/`
- `POST /api/books/`
- `GET /api/books/{id}/`
- `PATCH /api/books/{id}/`
- `DELETE /api/books/{id}/`

Supported list query parameters:

- `search`
- `genre`
- `author`
- `language`
- `year_min`
- `year_max`
- `ordering`
- `page`
- `page_size`

Permissions:

- `GET` endpoints are public
- `POST`, `PATCH`, and `DELETE` are staff-only

Example response:


```
{
  "count": 1,
  "next": null,
  "previous": null,
  "results": [
    {
      "id": 1,
      "title": "Deep Work",
      "language": "en",
      "publication_year": 2016,
      "authors": [
        {
          "id": 1,
          "name": "Cal Newport"
        }
      ],
      "genres": [
        {
          "id": 1,
          "name": "Productivity"
        }
      ],
      "average_external_rating": 4.5,
      "external_rating_count": 850
    }
  ]
}
```

Authors

- `GET /api/authors/`
- `POST /api/authors/`
- `GET /api/authors/{id}/`
- `PATCH /api/authors/{id}/`
- `DELETE /api/authors/{id}/`

Genres

- `GET /api/genres/`
- `POST /api/genres/`
- `GET /api/genres/{id}/`
- `PATCH /api/genres/{id}/`
- `DELETE /api/genres/{id}/`

Catalog write permissions:

- `POST`, `PATCH`, and `DELETE` for books, authors, and genres require an authenticated staff user.

Bookshelf Endpoints

These endpoints require authentication.

- `GET /api/me/bookshelf/`
- `POST /api/me/bookshelf/`
- `GET /api/me/bookshelf/{id}/`
- `PATCH /api/me/bookshelf/{id}/`
- `DELETE /api/me/bookshelf/{id}/`

Example request:

```
{
  "book_id": 1,
  "status": "reading",
  "personal_rating": 5,
  "is_favorite": true,
  "notes": "Strong first impression."
}
```

Example response:

```
{
  "id": 1,
```

```
  "book": {
    "id": 1,
    "title": "Atomic Habits"
  },
  "status": "reading",
  "personal_rating": 5,
  "is_favorite": true,
  "notes": "Strong first impression."
}
```

Review Endpoints

- `GET /api/books/{id}/reviews/`
- `POST /api/books/{id}/reviews/`
- `PATCH /api/reviews/{id}/`
- `DELETE /api/reviews/{id}/`

Example request:

```
{
  "rating": 5,
  "title": "Excellent",
  "content": "Focused, practical, and easy to apply.",
  "contains_spoiler": false
}
```

Example response:

```
{
  "id": 1,
  "user": 1,
  "book": 1,
  "rating": 5,
  "title": "Excellent",
  "content": "Focused, practical, and easy to apply.",
  "contains_spoiler": false
}
```

Recommendation Endpoints

Personalized Recommendations

- `GET /api/recommendations/for-me/`

Behavior:

- uses personal bookshelf ratings and review ratings
- excludes books already present in the user's bookshelf or review history
- returns human-readable recommendation reasons

Example response:

```
{
  "results": [
    {
      "book_id": 2,
      "title": "Strategic Thinking",
      "score": 0.86,
      "reasons": [
        "Matches your preferred genres.",
        "Shares an author with books you rated highly."
      ]
    }
  ]
}
```

Similar Books

- `GET /api/recommendations/similar-books/{book_id}/`

Analytics Endpoints

- `GET /api/analytics/books/trending/`
- `GET /api/analytics/genres/popularity/`
- `GET /api/analytics/ratings/distribution/`
- `GET /api/analytics/me/reading-summary/`
- `GET /api/analytics/authors/top/`

Example analytics response:

```
{
  "results": [
    {
      "genre": "Productivity",
      "book_count": 12
    },
    {
      "genre": "Strategy",
      "book_count": 8
    }
  ]
}
```

Response Formats

Common response patterns:

- list endpoints use paginated JSON with `count`, `next`, `previous`, and `results`
- detail endpoints return a single JSON object
- analytics endpoints return either a `results` array or a compact metrics object
- write endpoints return the created or updated resource payload

Representative metrics response:

```
{
  "completed": 4,
  "reading": 1,
  "favorites": 2,
  "reviews_written": 3,
  "average_personal_rating": 4.5
}
```

Error Handling

The API returns structured JSON errors.

Example response:

```
{
  "error": {
    "code": "validation_error",
    "message": "Validation failed.",
    "details": {
      "book_id": [
        "You already have a bookshelf entry for this book."
      ]
    }
  }
}
```

Status Code Conventions

- `200 OK`
- `201 Created`
- `204 No Content`
- `400 Bad Request`
- `401 Unauthorized`
- `403 Forbidden`
- `404 Not Found`

Google Books Data Import

The catalog is designed to ingest Google Books metadata into the local SQLite database.

Typical workflow:

1. Fetch Google Books payloads.
2. Normalize titles, authors, genres, ISBNs, and metadata.
3. Store them in local tables.

4. Build recommendations and analytics from local data plus user activity.

Management command examples:

```
python manage.py import_google_books --input-file data/samples/google_books_raw_sample.json
python manage.py import_google_books --query "productivity" --pages 2 --max-results 20
```

OpenAPI Schema

The machine-readable schema is exported to `docs/api_openapi.yaml`.